

Web Sanitiser in Rust - Relazione finale

Programmazione di Sistema, Politecnico di Torino

Filippo Barboni (s352701) · Marika Fuccio (s354828) · Alessio Belluardo (s359092)

1. Analisi dei requisiti

1.1 Il problema

I contenuti presenti sul web sono una delle vie principali usate dagli attacchi informatici per infettare i computer. Tra le minacce più comuni troviamo:

- Script dannosi e codice nascosto;
- Link e reindirizzamenti pericolosi;
- Download automatici (drive-by);
- Elementi di tracciamento.

Un sanitiser si colloca fra il contenuto grezzo non fidato e l'utente, e ha il compito di **neutralizzare** ciò che è pericoloso restituendo una versione utilizzabile del documento, non di limitarsi a segnalarlo o a rifiutarlo in blocco.

La differenza rispetto a un antivirus è sostanziale ed è dichiarata nella sezione 9 della traccia: non c'è analisi dinamica, non c'è esecuzione in sandbox, non c'è euristica comportamentale. Lo strumento ragiona su ciò che vede nei byte, applicando regole dichiarate e configurabili.

1.2 Requisiti funzionali (sez. 3)

Requisito	Dove è realizzato
Accettare file locali, alberi di directory e URL remoti	<code>input::classify_arg</code> , <code>input::file::expand_dir</code>
Riconoscere il tipo reale del contenuto senza fidarsi della dichiarazione	<code>policy::rules::sniff_mime</code> , <code>mime_mismatch</code>
Rimuovere costrutti attivi dall'HTML	<code>parser::rewriter::sanitise_html</code>
Ispezionare ogni URL del documento e agire secondo policy	<code>parser::rewriter::analyse_url</code> , <code>parser::dom::extract_links</code>
Ripulire i fogli di stile	<code>parser::css::sanitise_css</code>
Fetch opzionale e limitato delle sottorisorse	<code>input::subresource::crawl_subresources</code>
Produrre contenuto ripulito e report leggibile da programma	<code>engine::worker::write_output</code> , <code>report::json_emitter</code>
Modalità batch e policy configurabili da file	<code>main.rs</code> , <code>policy::SanitiserPolicy::load</code>

1.3 Requisiti non funzionali (sez. 4)

La sezione 4 impone quattro vincoli che hanno guidato molte scelte di dettaglio, e che vale la pena elencare perché ciascuno ha lasciato una traccia precisa nel codice.

Nessun panic sul parsing. L'input è controllato dall'attaccante: un panic non è un guasto, è una vulnerabilità di denial of service. Il crate non contiene `unwrap()` su dati provenienti dall'esterno, e ogni fallimento è un valore `Result` che il chiamante è obbligato a gestire.

Nessuna allocazione non limitata. Vale per entrambe le porte d'ingresso: la lettura da disco verifica la dimensione sui metadati **prima** di leggere, la lettura da rete controlla il tetto dentro il ciclo dei chunk, prima di concatenare. In nessuno dei due casi un input da diversi GB arriva mai in memoria.

Budget di tempo e di dimensione per input. `max_processing_ms` e `max_input_bytes`, verificati a checkpoint fra le fasi della pipeline.

Il throughput deve scalare con i core. Il numero di worker è `available_parallelism()` per default ed è configurabile da riga di comando, scelta che serve al funzionamento normale ma anche, e soprattutto, a poter tracciare le curve di speed-up richieste dalla sezione 7.

1.4 Quali attacchi abbiamo deciso di affrontare

Abbiamo preso come perimetro l'elenco esplicito della traccia, e per ciascuna voce esiste una difesa identificabile nel codice:

Attacco	Difesa
XSS via script, inline event handlers, <code>javascript:</code> and <code>data:</code> URI	rimozione nel rewriter
Active embedded content (<code>iframe</code> , <code>object</code> , <code>embed</code>)	rimozione salvo allow-list
Automatic client-side redirect (<code>meta refresh</code>)	rimozione della direttiva
MIME Confusion	sniffing dei magic bytes contro il tipo dichiarato
Decompression Bomb	decompressione in streaming con tetto sui byte
XML Bomb	conteggio delle espansioni con tetto configurabile
Huge Images	Rifiuto sulle dimensioni dichiarate nell'intestazione, senza allocare l'immagine in memoria
PDF active content	ricerca dei marcatori di azione automatica
SSRF, including via redirect chains	risoluzione e rivalidazione a ogni hop
Unicode confusion in URLs (IDN homographs, host/split)	normalizzazione e marcatura

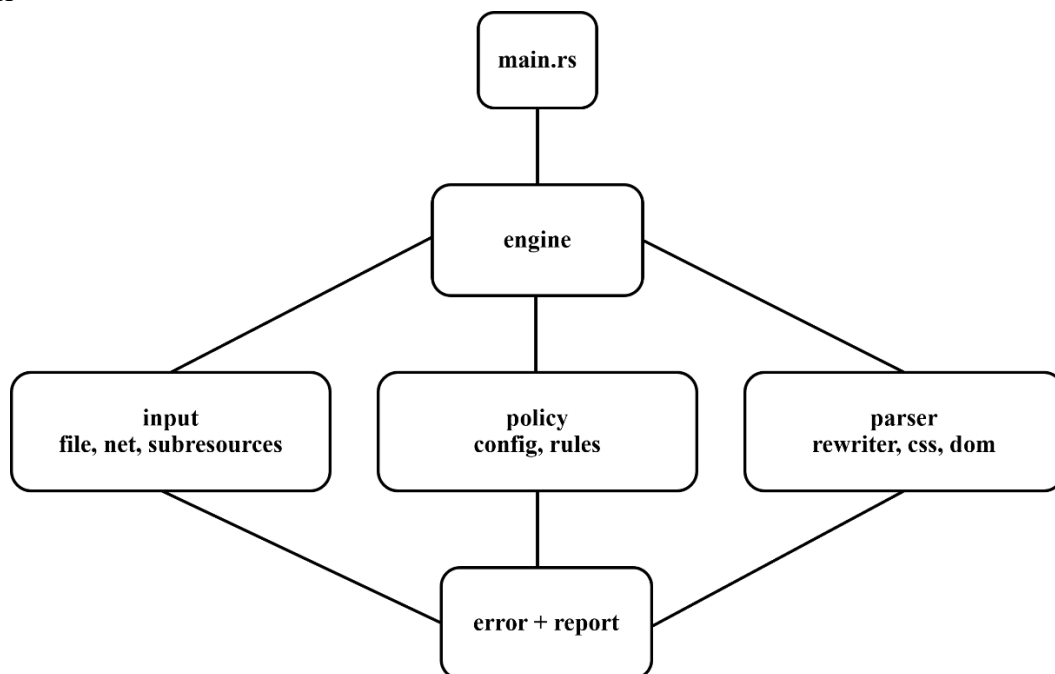
2. Architettura

2.1 Libreria e binario

La sezione 6 chiede che la separazione fra crate binario e crate libreria sia netta, «*so that the engine is reusable and independently testable*». Il manifesto dichiara due target distinti: la libreria `web_sanitiser` contiene il motore per intero, il binario `web-sanitiser` è un file di poco più di cento righe che analizza gli argomenti, carica la policy, chiama `run_sanitisation_pipeline` e stampa il report.

Questa separazione non è pura estetica: è ciò che rende i test di integrazione possibili senza avviare processi. Un test in `tests/` costruisce direttamente i `Source`, chiama la pipeline e legge il `Report`, senza mai passare per `main`. È anche la ragione per cui i benchmark misurano ciò che ci interessa e non l'avvio del processo.

2.2 I sei strati



Le dipendenze vanno in una sola direzione, dal basso verso l'alto: `error` e `report` non conoscono nessuno, `policy` conosce solo loro, `parser` conosce `policy` e `report`, `input` conosce `parser` e `policy`, `engine` conosce tutto. Nessun ciclo.

2.3 Il percorso di un input

1. **Classificazione:** la stringa passata con `-i` diventa `Source::File` o `Source::Url`; una directory viene espansa ricorsivamente
2. **Accodamento:** tutti i `Source` finiscono in una coda condivisa
3. **Caricamento:** il worker legge da disco o scarica dalla rete, applicando il tetto di dimensione **prima** di allocare
4. **Rifiuti duri:** tipo attivo dichiarato, XML bomb, huge images, PDF con contenuto attivo: di questi non esiste una versione ripulita
5. **Sniffing:** si determina il tipo reale e si registra l'eventuale discrepanza col dichiarato
6. **Sanitizzazione per tipo:** HTML nel rewriter, CSS nel suo ripulitore, il resto pass-through dopo i controlli sui byte
7. **Crawl opzionale:** solo per input URL e solo se abilitato
8. **Scrittura:** output ripulito su disco, report al thread principale

Il punto 4 precede il 5 e il 6 di proposito: un rifiuto costa un confronto, una sanitizzazione costa una scansione del documento. Non ha senso pagare la seconda per un input che verrà comunque respinto.

3. Il manifesto: **Cargo.toml** e le dipendenze

Il manifesto non è un dettaglio burocratico: dichiara la superficie di fiducia del progetto. Ogni dipendenza è codice di terzi che elabora byte ostili, quindi ciascuna è stata scelta con un criterio esplicito.

3.1 Il pacchetto e i due target

```
[package]
name = "web-sanitiser"
version = "0.1.0"
edition = "2021"
```

```
[lib]
name = "web_sanitiser"
path = "src/lib.rs"

[[bin]]
name = "web-sanitiser"
path = "src/main.rs"
```

I due nomi differiscono per un carattere e la differenza è voluta: **underscore** per la libreria, perché è l'identificatore che si scrive nel codice (use `web_sanitiser::...`), e **trattino** per il binario, perché è quello che si digita nel terminale. Cargo non permette di usare lo stesso nome per entrambi, e provare a forzarlo produce un conflitto di percorsi in `target/`.

3.2 Le dipendenze, una per una

```
clap = { version = "4", features = ["derive"] }
```

Analisi degli argomenti da riga di comando. La feature `derive` permette di descrivere l'interfaccia come una struct annotata invece che come una sequenza di chiamate: i doc-comment dei campi diventano il testo di `-help`, quindi la documentazione dell'interfaccia non può divergere dall'interfaccia stessa. È l'unica dipendenza usata soltanto dal binario, mai dalla libreria.

```
serde = { version = "1", features = ["derive"] }
serde_json = "1"
```

`serde` è il framework di serializzazione, `serde_json` il suo formato JSON. Servono in due punti opposti: **leggere** il file di policy e **scrivere** il report. La feature `derive` genera le implementazioni a compile time a partire dalle nostre struct, quindi il formato del report è definito dalla forma dei tipi e non da codice di serializzazione scritto a mano, che potrebbe divergere.

L'attributo `#[serde(default)]` su `SanitiserPolicy` fa sì che un file di configurazione **parziale** sia valido: i campi assenti mantengono il valore predefinito. È ciò che rende utilizzabile una policy di tre righe.

```
url = "2"
```

Analizza e normalizza gli URL come fa un browser, seguendo lo stesso standard. Non è una comodità: è una **difesa**. Scrivere un parser di URL a mano significa introdurre proprio quelle divergenze di interpretazione che gli attacchi `host/split` sfruttano. Il crate applica anche la conversione degli hostname Unicode in punycode, che è il motivo per cui il rilevamento degli omografi funziona guardando un prefisso `xn--`.

```
lol_html = "1.2"
```

Il rewriter HTML. È la scelta architetturale più consequenziale del progetto e merita la discussione estesa in sez. 4.4.

```
tokio = { version = "1", features = ["rt-multi-thread", "time", "net"] }
```

Il runtime asincrono che ospita le chiamate di rete. Le tre feature sono selezionate, non prese in blocco: `rt-multi-thread` per l'esecutore, `time` per i timeout, `net` per i socket. Manca deliberatamente `macros`: non usiamo `#[tokio::main]` né `#[tokio::test]`, perché il runtime lo costruiamo esplicitamente nello scheduler e vogliamo che sia visibile dove nasce e dove muore.

```
request = { version = "0.12", default-features = false, features = ["rustls-tls", "gzip"] }
```

`default-features = false` disattiva l'intero insieme predefinito, che includerebbe fra l'altro il supporto ai cookie e il backend TLS di sistema. È una scelta di sicurezza attiva: **senza cookie store il client non può inoltrare credenziali**, e questa è la difesa contro la fuga di sessione dello scenario `echo-headers`. La difesa esiste per costruzione, non per una riga di codice che qualcuno potrebbe rimuovere.

`rustls-tls` sceglie un'implementazione TLS scritta in Rust invece di OpenSSL: niente dipendenza di sistema da installare, e la superficie di attacco crittografica resta in codice con le stesse garanzie di memoria del nostro.

`gzip` abilita la decompressione trasparente in streaming. Questa scelta ha una conseguenza importante e discussa in sez. 6.4.

```
imagesize = "0.15.0"
```

Legge larghezza e altezza di un'immagine dalla sua intestazione, **senza mai decomprimerla in memoria**. È esattamente la difesa contro le dimension bomb: un PNG di ottanta byte che dichiara `65535×65535` richiederebbe circa 12 GB per essere decodificato, e noi lo rifiutiamo avendo letto solo i primi byte.

Abbiamo valutato di scrivere a mano i parser di intestazione dei quattro formati che riconosciamo. Li abbiamo scartati per un motivo di rischio: quattro parser di intestazioni binarie scritti da noi, su input ostile, sono quattro superfici per un bug, mentre la dipendenza è piccola, a scopo unico e senza dipendenze transitive a runtime.

3.3 Il profilo di release

```
[profile.release]
opt-level = 3
lto = true
```

`opt-level = 3` è l'ottimizzazione massima. `lto = true` abilita la link-time optimization, che permette al compilatore di ottimizzare attraverso i confini dei crate: senza, le funzioni di `lol_html` e `reqwest` resterebbero scatole chiuse. Il profilo esiste perché le misure della sezione 7 vanno prese solo su build di release. Su una build di debug si misurerebbero i controlli a runtime del compilatore, overflow degli interi, indici fuori dai limiti, che in release non ci sono.

4. I moduli

4.1 `error`: il modello di errore

Un solo enum per tutto il crate, con otto varianti, ciascuna corrispondente a una **categoria di causa** e non a un punto del codice:

Variante	Significato
<code>Io</code>	errore del file system
<code>InvalidUrl</code>	URL malformato
<code>Fetch</code>	guasto di rete: DNS, connessione, TLS
<code>SsrfBlocked</code>	destinazione interna, fermata dalla guard
<code>Config</code>	policy o report non leggibili
<code>BudgetExceeded</code>	sforato un limite dichiarato
<code>Refused</code>	contenuto respinto dalla policy
<code>Parse</code>	fallimento nella riscrittura del documento

`Display` e `std::error::Error` sono scritti a mano invece di derivarli con `thiserror`. La ragione è didattica ma anche pratica: il modello di errore resta tutto visibile in un file solo, senza macro da espandere mentalmente per capire che messaggio uscirà.

Le implementazioni di `From` sono ciò che rende utilizzabile l'operatore `?`: un `std::io::Error` diventa `SanitiserError::Io` automaticamente, senza un `map_err` a ogni chiamata.

La conversione da `reqwest::Error` è il punto più interessante del modulo, perché non è una traduzione meccanica ma una **classificazione**:

```

if e.is_decode()      { Refused(...) }      // contenuto non ispezionabile
else if e.is_timeout() { BudgetExceeded(...) } // budget dichiarato dalla policy

else if e.is_redirect() { Refused(...) }      // decisione della nostra policy
else                    { Fetch(...) }        // guasto vero

```

La distinzione conta perché a valle determina lo **stato nel report** e quindi l'**exit code della CLI**. Un redirect interrotto dalla nostra redirect policy è una decisione voluta, non un guasto: se finisse in **Fetch** verrebbe riportato come **Error**, e siccome l'exit code 1 dipende solo dai rifiuti, uno scenario SSRF bloccato uscirebbe con codice 0, cioè apparirebbe come un successo.

4.2 report: l'esito verificabile

Il report è metà del prodotto: la sezione 3 chiede contenuto ripulito e resoconto machine-readable. La struttura riflette la scelta di rendere ogni modifica verificabile da chi legge e non solo dal tool: un'**Action** porta quattro informazioni — quale regola è scattata, dove, cosa c'era prima e cosa c'è adesso.

JobStatus ha tre valori e la distinzione è semantica:

- **Sanitised**: ripulito, con o senza modifiche
- **Refused**: respinto in blocco: sfiora un limite, lo vieta la policy, oppure non è ispezionabile. Ne basta uno per far uscire la CLI con 1
- **Error**: non è stato possibile elaborarlo. Non tocca l'exit code, perché un input illeggibile non deve far fallire l'intero lotto

I due campi opzionali **refusal_reason** ed **error** si escludono a vicenda e spariscono dal JSON quando sono vuoti, grazie a `#[serde(skip_serializing_if = "Option::is_none")]`. È una promessa che si può rompere per sbaglio togliendo un attributo senza che nulla si accorga a compilazione, quindi è fissata da un test dedicato.

json_emitter.rs è il sottomodulo che scrive: serializza il **Report** su file oppure su standard output. La separazione fra le strutture dati e la loro emissione tiene fuori dal modello ogni preoccupazione di formattazione, e significa che aggiungere in futuro un secondo formato, un riepilogo leggibile da persona, per esempio, non richiederebbe di toccare i tipi.

Report::assemble riceve i conteggi già calcolati da chi ha eseguito la pipeline invece di ricavarli scorrendo i job. È una scelta che evita di contare due volte la stessa cosa: i contatori atomici hanno già il totale, e ricalcolarlo qui aprirebbe la possibilità che le due strade divergano.

4.3 policy: configurazione e regole

config.rs

Ventuno campi, tutti con un default sensato e tutti sovrascrivibili da file.

Una alternativa che abbiamo considerato. Il termine «policy» copre oggi due cose diverse: i **requisiti funzionali**, cosa fare con un link sospetto, quali origini sono fidate, e i **requisiti non funzionali**, quanto grande può essere un input, quanto tempo può durare l'elaborazione. Si sarebbero potute separare in due struct, **Policy** e **Limits**, rendendo la distinzione visibile nel tipo. Abbiamo tenuto una struct sola perché il file di configurazione resta uno, e due tipi avrebbero richiesto due sezioni annidate nel JSON per un guadagno di sola chiarezza. È un compromesso, e lo segnaliamo.

Una seconda alternativa. **allow_loopback** e **block_private_addresses** sono due booleani che governano lo stesso asse, e con essi convive **fetch_host_allowlist**. Un modello più pulito sarebbe stato un unico insieme di classi di indirizzo consentite, `{private, loopback, link_local}`, invece di due interruttori indipendenti che possono essere combinati in modi che non significhino niente. La forma attuale è più immediata da leggere in un JSON, ma espone quattro combinazioni di cui solo tre hanno senso.

La differenza fra `fetch_host_allowlist` e `allow_loopback` merita di essere chiarita perché si sovrappongono solo in apparenza. Il primo è un elenco di **host nominali** dichiarati fidati; il secondo è una deroga su una **classe di indirizzi**. Un host in allow-list è esentato dai controlli anche se risolve a un indirizzo privato qualunque; `allow_loopback` invece consente il solo loopback, per qualunque host che ci risolva. Si sarebbe potuto esprimere il secondo col primo, mettendo `localhost` e `127.0.0.1` in allow-list, ma si sarebbe persa la possibilità di consentire il loopback per host non elencati.

`reject_active_types` rifiuta un contenuto il cui MIME **dichiarato** è di tipo script, indipendentemente da cosa contenga il corpo. Può sembrare eccessivo: rifiutiamo del testo innocuo se qualcuno lo etichetta `text/javascript`. Ma è la lettura corretta della minaccia, ed è ciò che lo scenario `text-disguised-asjavascript` verifica: chi riceve quel file lo eseguirà **per come è dichiarato**, non per quello che contiene. Una dichiarazione non si può ripulire, quindi l'unica risposta possibile è rifiutare.

rules.rs

Il modulo delle regole di ispezione: funzioni pure, senza stato, che guardano byte o stringhe e rispondono a una domanda.

Sniffing e dichiarato. `sniff_mime` riconosce il tipo reale dai magic bytes: firme binarie forti per PNG, JPEG, GIF, WebP, PDF, ZIP e gzip, e per il testo una ricerca nei primi 512 byte. `mime_mismatch` confronta il dichiarato con lo sniffato e produce un'azione, non un rifiuto: una discrepanza è un fatto da riportare, non necessariamente un attacco.

`is_html` combina i due in modo asimmetrico, e l'asimmetria è la parte interessante: se lo sniffing ha riconosciuto una **firma binaria certa**, un PDF, un PNG, quel contenuto non è HTML qualunque cosa dica il `Content-Type`. Il dichiarato torna a contare solo quando lo sniffing non ha trovato nulla di deciso. È la regola che rende corretti entrambi i versi della confusion MIME: HTML travestito da immagine viene ripulito, PDF travestito da HTML no.

XML bomb. `detect_xml_bomb` estrae le dichiarazioni di entità, ne costruisce il grafo delle dipendenze e conta le espansioni riusando i conteggi già calcolati fermandosi a un tetto di profondità di ricorsione: senza, contare quante volte un'entità si espande costerebbe quanto espanderla davvero, cioè proprio l'esplosione da cui ci stiamo difendendo. Il tetto serve a evitare che un'espansione ostile faccia esaurire lo stack proprio mentre sta cercando di proteggersi da un'espansione ostile.

SSRF e confusione degli host. `host_is_internal` copre loopback, RFC-1918, link-local, unspecified e broadcast, sia IPv4 sia IPv6, e riconosce le codifiche alternative degli indirizzi, la forma decimale `2130706433` è `127.0.0.1`. `url_has_host_confusion` cerca punteggiatura fullwidth, userinfo ingannevole e backslash come separatore, cioè i costrutti su cui parser diversi danno risposte diverse. `host_is_punycode` riconosce gli omografi sfruttando il fatto che il crate `url` ha già normalizzato l'hostname.

PDF e immagini. `pdf_has_active_content` cerca tre marcatori (`/JavaScript`, `/JS`, `/Launch`) solo su file che cominciano con `%PDF-`. Inizialmente ne includeva altri due, `/OpenAction` e `/AA`, tolti perché altrimenti facevano rifiutare qualunque PDF prodotto: in un documento normale quei marcatori indicano semplicemente quando qualcosa accade e non cosa. `image_too_large` delega a `imagesize` la lettura dell'intestazione.

4.4 parser: la sanitizzazione

rewriter.rs e la scelta dello streaming

La traccia parla di ispezionare il DOM. La nostra implementazione adotta un modello **equivalente ma diverso**: `lol_html` è un rewriter in streaming che non costruisce mai l'albero del documento. Attraversa i byte una volta sola, invoca un handler per ogni elemento incontrato, e riemette il risultato.

Il vantaggio è il consumo di memoria: un documento da 4 MB non genera un albero di nodi da decine di MB, e i dati sperimentali del benchmark lo confermano.

Il limite va dichiarato: **tutte le regole che applichiamo sono locali all'elemento**. Rimuovere uno `<script>`, cancellare un attributo `onclick`, riscrivere un `href` sono decisioni che si prendono guardando un solo nodo. Una trasformazione che richiedesse una visione globale dell'albero, “rimuovi ogni `<div>` che non contiene testo”, o una regola che dipende dal

fratello precedente, non sarebbe esprimibile in questo modello. Per l'insieme di regole che la traccia richiede la limitazione non intacca.

Lo smistamento. Un solo handler registrato sul selettore `*` intercetta ogni elemento e smista: prima toglie ogni attributo che comincia per `on`, poi distingue per tag. Gli script vanno rimossi salvo `allow-list`; il contenuto incorporato (`iframe`, `object`, `embed`, `frame`) viene rimosso e il nome dell'azione registrata dipende da **cosa** aveva di sbagliato la sorgente; i **meta** con `http-equiv=refresh` perdono la direttiva. Per tutti gli altri elementi si passa agli attributi che portano URL.

Le tre modalità di LinkAction. Un link sospetto può essere rimosso (**Remove**), sostituito da un segnaposto (**Placeholder**) o riscritto in forma inerte conservando l'originale (**Rewrite**). La terza modalità ha una conseguenza da conoscere: il valore prodotto contiene ancora l'URL originale come testo, quindi un documento sanificato in modalità **Rewrite** conserva l'hostname Unicode di un homograph, inerte, ma presente. Quello scenario richiede che l'hostname grezzo non sopravviva, quindi la conformità con il comportamento atteso vale con il default **Placeholder** e non con **Rewrite**.

Lo stato condiviso fra gli handler usa `Rc<RefCell<Vec<Action>>>` e non `Arc<Mutex<...>>`. La scelta è deliberata e va spiegata perché a prima vista sembra una svista in un progetto concorrente: la riscrittura di un singolo documento avviene **interamente dentro un thread**. Il parallelismo del progetto è fra input diversi, non dentro lo stesso input. Usare `Arc<Mutex>` qui significherebbe pagare un'atomica e un lock per ogni azione registrata, senza che nessun altro thread possa mai contendere quella struttura. `Rc` e `RefCell` sono i tipi giusti proprio perché **non** sono thread-safe: il compilatore impedirebbe di spostarli fra thread, e quel divieto documenta l'invariante.

css.rs

Tre costrutti da neutralizzare: `javascript:` dentro `url()`, la chiamata `expression()` di memoria Internet Explorer, e `@import`, che permette una catena di esfiltrazione. Non è un parser CSS completo ma una pulizia mirata, e la sezione 9 ci esonera dal battere l'offuscamento avanzato.

Vale la pena notare `replace_ci` e `strip_import`: avanzano per **caratteri** e non per byte (`ch.len_utf8()`), perché in UTF-8 un carattere può occuparne più di uno e un avanzamento a byte spezzerebbe le sequenze multibyte.

dom.rs

Estrae i valori degli attributi che portano URL (`href`, `src`, `action`, `data`, `poster`, `formaction`) senza modificare il documento. Riusa `lol_html` come tokenizzatore: dove serve solo leggere, non c'è motivo di avere un secondo parser con un secondo insieme di comportamenti sui casi limite.

4.5 input: l'astrazione sugli ingressi

mod.rs e file.rs

`classify_arg` è la biforcazione iniziale: `http://` e `https://` diventano `Source::Url`, tutto il resto `Source::File`. `expand_dir` scende ricorsivamente in una directory con uno stack esplicito invece che per ricorsione (su un albero molto profondo la ricorsione esaurirebbe lo stack) e ignora i collegamenti simbolici, che potrebbero creare cicli o portare fuori dall'albero di partenza.

`read_file` verifica la dimensione sui **metadati** prima di leggere. Il controllo dopo la lettura rifiuterebbe correttamente ma solo dopo aver allocato l'intero file: su un input da diversi GB il rifiuto arriverebbe dopo il danno, in contraddizione con la sezione 4.

network.rs

`build_client` costruisce il client una volta sola per l'intera esecuzione. `request::Client` contiene il pool di connessioni e la configurazione TLS: è progettato per essere condiviso, e `clone()` incrementa un contatore invece di duplicare il pool. Ricostruirlo a ogni richiesta getterebbe via il keep-alive e imporrebbe un nuovo handshake TLS anche verso host già contattati.

ssrf_guard risolve l'host e rifiuta le destinazioni interne prima di aprire la connessione.

build_redirect_policy rivalida ogni hop con lo stesso criterio: un redirect verso un indirizzo interno viene bloccato, non seguito ciecamente. È la difesa contro il bypass SSRF via catena di redirect.

Le due funzioni condividono la logica di valutazione, e ne deriva una **leggera duplicazione di codice** che abbiamo scelto di tenere. La ragione è che unificarle richiederebbe di chiamare `to_socket_addrs()`, una risoluzione DNS, quindi un'operazione potenzialmente bloccante e costosa, anche nei casi in cui i controlli testuali bastano a decidere. Preferiamo due percorsi leggermente ridondanti a un percorso unico che paga il resolver più spesso del necessario.

Sulla cache degli URL risolti che la sezione 5.1 cita fra gli stati condivisi: non l'abbiamo implementata, e non per dimenticanza. Sotto `to_socket_addrs()` c'è il resolver del sistema operativo, che una cache la tiene già per conto suo, con politiche di scadenza corrette rispetto ai TTL del DNS. Una nostra cache in memoria duplicherebbe quel lavoro aggiungendo uno stato condiviso da sincronizzare, e uno stato che, se non rispettasse i TTL, sarebbe pure una vulnerabilità: un record scaduto potrebbe far contattare un indirizzo che nel frattempo è cambiato.

La lettura del corpo avviene a blocchi, e il tetto `max_input_bytes` è verificato **prima** di concatenare ogni blocco.

Sulla decompressione: gestiamo solo `gzip`, non `br` né `zstd`. Le codifiche che non sappiamo decodificare vengono rifiutate, perché dichiarare “sanitizzato” un contenuto che non siamo in grado di leggere sarebbe falso. Un'estensione futura può abilitare le altre due: `reqwest` le supporta dietro feature, e la logica di rifiuto resterebbe la stessa per ciò che resta fuori.

subresource.rs

Il crawl parte dall'**HTML già sanificato**, e questa è la sua proprietà più importante: segue solo i riferimenti sopravvissuti alla riscrittura. I link pericolosi sono già stati sostituiti da segnaposto, che `absolute_links` scarta. Non è quindi una seconda linea di difesa contro i link cattivi, ma uno strumento per guardare cosa c'è dietro i riferimenti **leciti**.

I tre limiti richiesti dalla sezione 3 sono contatori che si consumano: profondità, byte totali e numero di richieste. A questi abbiamo aggiunto un checkpoint sul budget di tempo, verificato a ogni giro prima di aprire una nuova connessione, perché il crawl è la fase più lunga dell'elaborazione.

Un insieme `visited` impedisce di scaricare due volte lo stesso URL: è ciò che spezza i cicli di auto-inclusione, prima ancora che intervenga il tetto di profondità.

Il crawl è attivo **solo per input di tipo URL**. Per un file locale mancherebbe una base per risolvere i riferimenti relativi, e soprattutto analizzare un file non deve generare traffico verso destinazioni scelte dal suo contenuto: chi ci consegna un HTML sospetto non deve poter dedurre, da una richiesta in uscita, che lo stiamo esaminando.

4.6 engine: il motore concorrente

mod.rs

Contiene due strutture, separate secondo un criterio preciso: **l'immutabilità**.

`WorkerContext` raccoglie tutto ciò che un worker riceve una volta e che poi non cambia più (policy, client HTTP, insieme dei nomi ambigui, directory di uscita). Siccome nessuno ci scrive, ogni worker può tenerne una copia **senza alcun lock**. Copiarla costa poco: i campi pesanti sono dietro `Arc` o, come il client, condivisi internamente.

`Statistics` raccoglie i contatori globali, che invece cambiano di continuo, e usa tipi atomici invece di un mutex.

Ordering::Relaxed è sufficiente perché gli incrementi sono **indipendenti fra loro**: non ci serve che i quattro contatori siano coerenti l'uno con l'altro a metà corsa, ci serve solo che nessun incremento vada perso. Un ordinamento più forte imporrebbe barriere di memoria che qui non danno un beneficio.

scheduler.rs

Il runtime **tokio** viene costruito con `new_multi_thread()` e i worker vi entrano con `block_on`. È un modello ibrido (thread di sistema per il lavoro CPU-bound, runtime asincrono per la rete) che non abbiamo visto a lezione e che quindi vale la pena giustificare. Il parsing e la riscrittura sono CPU-bound: metterli su task asincroni non guadagnerebbe niente e bloccherebbe l'esecutore. Le chiamate di rete sono I/O-bound: farle sincrone terrebbe un thread fermo ad aspettare. `block_on` è il ponte fra i due mondi: il worker, che è un thread vero, si sospende su una future finché la rete non risponde, mentre gli altri worker continuano.

La coda è un `Arc<Mutex<VecDeque<Source>>>` pre-riempita prima che i worker partano. È la ragione per cui non serve una `Condvar`: i worker non devono mai attendere che arrivi lavoro, perché tutto il lavoro c'è già. Ognuno prende, elabora e ritorna; quando la coda è vuota il ciclo termina. Una `Condvar` sarebbe necessaria in uno scenario a produttore-consumatore, dove il consumatore può trovare la coda vuota ma non finita (non è il nostro caso, e aggiungerla significherebbe pagare complessità per un problema che la struttura del programma non ha).

La sezione critica è minima: si prende il lock, si estrae un elemento, si rilascia. L'elaborazione avviene fuori. È ciò che permette al parallelismo di esprimersi: se il lock fosse tenuto durante la sanitizzazione, i worker sarebbero di fatto serializzati.

Il canale `mpsc` riporta i report al thread principale. Il `drop(tx)` dopo lo spawn dei worker è necessario: senza, resterebbe un mittente vivo nel thread principale e il ciclo `for report in rx` non finirebbe mai.

worker.rs

Il ciclo del worker e la gestione del poisoning meritano una nota. Se un thread va in panic mentre tiene un `Mutex`, quel mutex diventa *poisoned* e ogni successivo `lock()` restituisce un errore. Il comportamento predefinito sarebbe propagare il panic. Noi usiamo:

```
queue.lock().unwrap_or_else(|e| e.into_inner())
```

cioè recuperiamo comunque il dato. La scelta si giustifica sul tipo di dato: la coda contiene `Source`, valori inerti che un panic altrui non può aver reso incoerenti. Propagare il panic significherebbe far cadere tutti i worker perché uno solo ha avuto un problema su un input.

La scrittura dell'output è il punto in cui abbiamo dovuto scegliere fra due mali. La tentazione naturale sarebbe conservare il `PathBuf` completo dell'input e ricrearlo sotto la directory di uscita. Ma quel percorso viene da input non fidato, e ricostruirlo significherebbe permettere a un input di decidere dove scrivere: nel caso peggiore, sovrascrivere i file dell'utente. Il nome viene quindi ricostruito da zero, in modo che non possa contenere separatori di percorso.

Il prezzo è la perdita di informazione. Per i `file` si conserva il solo nome base, e la directory di provenienza sparisce: conta solo se due input sono omonimi e solo in quel caso, si aggiunge un suffisso derivato dal percorso completo. Per gli `URL` la sanificazione cancella schema, porta e query string, quindi URL diversi collaserebbero di continuo sullo stesso nome: lì il suffisso serve sempre. L'estensione viene dal tipo reale sniffato, non da quello dichiarato, perché il file salvato deve dire la verità su cosa contiene.

4.7 main.rs: la CLI

Poco più di cento righe: argomenti, policy, espansione delle directory, chiamata alla pipeline, emissione del report, exit code.

`available_parallelism()` determina il numero di worker predefinito. La sezione 4 chiede che il throughput scali con i core disponibili, e questa funzione restituisce i core visibili al processo, che dentro un container o su una macchina condivisa sono meno di quelli fisici ed è il numero corretto da usare. Il fallback a 4 copre il caso in cui il sistema non sappia rispondere.

Il numero di thread è configurabile con `-t`, e non solo per completezza: è il parametro che permette di tracciare le curve di speed-up richieste dalla sezione 7. Senza quella opzione la valutazione di scalabilità non sarebbe misurabile dall'esterno.

Gli `exit code` distinguono tre situazioni: `0` tutto sanificato, `1` almeno un input rifiutato, `2` errore d'uso. L'`1` non è un errore del programma ma il modo in cui la CLI comunica a uno script che qualcosa è stato respinto.

4.8 `scripts/benchmark.py`: lo strumento di misura

La sezione 8 elenca fra i deliverable «*the test corpus and benchmark scripts used for the evaluation*»: lo strumento di misura è parte della consegna, perché è ciò che rende la valutazione riproducibile da chi legge.

Lo script è in Python e non in Rust. La ragione decisiva sono i **grafici**: la sezione 7 li vuole fra i risultati, e in Rust non esiste nulla di paragonabile a matplotlib (le librerie che ci sono produrrebbero cinque figure leggibili ma con un costo di codice più codice molto maggiore).

Un framework di microbenchmark come `criterion` sarebbe stato la scelta idiomatica in Rust, ma misura funzioni in-process: non risponde a nessuna delle quattro voci della sezione 7, che riguardano tutte il comportamento del programma completo.

Inoltre, usando Rust, misurare la memoria **allocata dal nostro codice**, invece di quanta ne occupi il processo nel suo complesso, richiederebbe un allocatore globale strumentato: il tratto `GlobalAlloc` è `unsafe` per definizione, e sia `lib.rs` sia `main.rs` portano `#![forbid(unsafe_code)]`.

Lo script genera i propri input sintetici, esegue il binario di release, raccoglie i tempi e il picco di memoria campionato, e produce sia i dati grezzi (`scripts/results.json`) sia le cinque figure. Le opzioni `--reps`, `--only`, `--quick` e `--origin` permettono di rieseguire una misura sola o di provare la catena senza attendere un giro completo. La documentazione metodologica sta nel `README.md`.

5. Aspetti di programmazione di sistema in Rust

5.1 Il modello di concorrenza

Il progetto usa quattro primitive, ciascuna scelta sull'uso reale:

Primitiva	Dove	Perché
<code>Arc<T></code> senza lock	policy, allow-list, nomi ambigui	nessuno scrive, quindi non serve escludere nessuno
<code>Arc<Mutex<VecDeque>></code>	coda dei job	tutti scrivono, e le operazioni devono essere atomiche
<code>AtomicU64</code>	contatori	ogni incremento è indipendente: il lock sarebbe sprecato
<code>mpsc::channel</code>	ritorno dei report	trasferimento di possesso, non condivisione

La distinzione fra le ultime due è quella che il corso insegna come discriminante: si condivide con un lock ciò che va letto e scritto da più parti, si **trasferisce** con un canale ciò che ha un solo destinatario. Un report prodotto da un worker non serve a nessun altro worker: passarlo su un canale significa che il possesso si sposta e nessuno deve più sincronizzarsi su di esso.

Il sistema dei tipi garantisce l'assenza di data race a compile time: un tipo non `Send` non può attraversare un confine di thread, e questo è il motivo per cui `Rc<RefCell>` dentro il rewriter è sicuro, non perché lo abbiamo verificato, ma perché il compilatore rifiuterebbe di compilare un uso scorretto.

5.2 Possesso, prestiti e zero-copy

La sezione 5.2 chiede di ridurre le copie. Le scelte principali:

Lo streaming del rewriter. `lol_html` non costruisce l'albero DOM: legge, trasforma e riemette. La memoria non è proporzionale alla struttura del documento ma alla finestra di lavoro.

`String::from_utf8_lossy` e il `Cow`. La funzione restituisce un `Cow<str>`, *clone on write*. Se i byte sono UTF-8 valido, il `Cow` è un prestito e **non viene copiato nulla**; solo se ci sono sequenze non valide viene allocata una `String` nuova con i caratteri di sostituzione. Sul corpus benigno, che è UTF-8 corretto, la conversione costa zero byte allocati; è solo sull'input ostile che si paga la copia, esattamente il compromesso giusto.

Il client HTTP condiviso. Un `request::Client` per l'intera esecuzione, clonato dai worker: il pool di connessioni e la configurazione TLS esistono una volta sola.

I prestiti nella pipeline. `WorkerContext` viene passato per riferimento a `process_one`, che estrae i riferimenti ai singoli campi. Nessuna copia della policy per input.

Un punto dove **non** abbiamo evitato la copia, e vale la pena dirlo: il conteggio dei nomi ambigui usa `to_string_lossy().to_string()`. Il `Cow` presterebbe i byte, ma la chiave di una `HashMap` deve essere posseduta, quindi la copia c'è. Avviene una volta per input all'avvio, quindi il costo è trascurabile, ma è un caso in cui il tipo di dato impone l'allocazione.

5.3 La gestione degli errori

Nessun `unwrap()` su dati esterni. Ogni funzione che può fallire restituisce `Result`, e l'operatore `?` propaga senza rumore grazie alle implementazioni di `From`.

La proprietà che ne consegue è quella richiesta dalla sezione 4: **un input malformato non fa cadere il programma**. Il worker cattura il fallimento, lo traduce in un `JobReport` con stato `Error`, e passa all'input successivo. Il batch continua.

5.4 `unsafe`

Zero blocchi `unsafe` in tutto il progetto. Sia `lib.rs` sia `main.rs` portano `#![forbid(unsafe_code)]`, che non è un consiglio ma un divieto imposto dal compilatore: `forbid` non è disattivabile nemmeno da un `allow` locale in un modulo interno.

La scelta ha un costo concreto che abbiamo accettato. Per misurare il picco di memoria attribuibile al nostro codice servirebbe un allocatore globale strumentato, e il tratto `GlobalAlloc` richiede `unsafe` per definizione. Abbiamo preferito misurare il picco di memoria residente **dall'esterno**, con `psutil` che campiona il processo, ottenendo un numero che include anche il runtime e l'allocatore ma senza incrinare la garanzia.

6. Verifica: test, corpus e ground truth

Il progetto contiene **62 test** distribuiti su tre livelli.

6.1 Test di unità (33)

Vivono dentro i moduli sorgente, in blocchi `#[cfg(test)] mod tests`, accanto al codice che provano. La collocazione non è stilistica: da lì raggiungono le funzioni **private**, che dall'esterno del crate sarebbero irraggiungibili. Diversi controlli di sicurezza sono privati proprio perché non fanno parte dell'interfaccia (`is_loopback_host` in `network.rs`, `absolute_links` in `subresource.rs`) e i loro test sono l'unico modo per esercitarli.

La distribuzione riflette dove sta la logica delicata: undici in `rules.rs`, quattro ciascuno in `config.rs`, `file.rs` e `report/mod.rs`, due in `network.rs`, `subresource.rs` ed `engine/mod.rs`.

I test più significativi sono quelli che fissano una **proprietà negativa**, cioè verificano che qualcosa *non* accada: che `localhost.evil.test` non sia riconosciuto come loopback, che i segnaposto del rewriter non vengano ri-caricati dal crawl, che un campo opzionale vuoto non compaia nel JSON.

6.2 `corpus_test.rs` e la ground truth (1 test, 24 casi)

Un solo `#[test]`, ma guidato dai dati: legge `corpus/ground_truth.json` ed esegue la pipeline su ogni campione elencato.

Il **corpus** contiene 24 campioni: 7 benigni e 17 maligni. I benigni includono casi deliberatamente insidiosi ma legittimi, una pagina con un **data**: URI di una vera immagine, un link a un dominio simile a uno noto ma innocuo, Unicode legittimo. Servono a misurare i falsi positivi, che senza campioni “difficili” sarebbero sempre zero e quindi privi di significato.

La **ground truth** è etichettata a mano da noi e dichiara, per ogni campione, l'esito atteso e le regole che devono comparire. Il file lo dice esplicitamente: le etichette **non sono usate dal tool a runtime**, ogni input è trattato come non fidato. Sono materiale di valutazione, non di esecuzione.

Il test accumula tutti i disallineamenti in un vettore e asserisce una volta sola alla fine, invece di fermarsi al primo. Una modifica che ne rompe cinque li mostra tutti e cinque insieme.

6.3 `evil_origin_test.rs` (28 test)

Verificano il comportamento contro il server ostile fornito dal docente. Sono marcati `#[ignore]` perché richiedono Docker: un `cargo test` normale li salta, e si lanciano con `-- --ignored`.

Coprono 26 dei 28 scenari del catalogo. I due mancanti, `/redirect/hop-two` e `/redirect/final-script`, non hanno un test proprio ma vengono attraversati dal sanitiser mentre segue la catena del triple-hop.

L'aspetto metodologicamente rilevante è che ogni scenario di evil-origin porta con sé un campo `expectedSanitizerBehavior`: **quello è la specifica** contro cui il test è scritto. Le asserzioni non sono state inventate, sono lette dal catalogo degli scenari.

Questo ha guidato la forma delle asserzioni. Dove l'oracolo chiede una proprietà **esaustiva** («*the sanitized output must contain no data: scheme in any attribute*»), il test conta le occorrenze invece di verificarne una sola. Dove l'oracolo nomina un **meccanismo** («*the parser must enforce an entity-expansion cap*») il test verifica la motivazione del rifiuto e non solo lo stato, perché un rifiuto per il motivo sbagliato sarebbe comunque superato.

6.4 Divergenze dichiarate rispetto agli scenari

Due scenari sono superati con un comportamento diverso da quello che gli scenari descrivono.

/mime/gzip-bomb. Il comportamento atteso è «*Reject response with Content-Encoding header before decompression*»: vedere l'header, rifiutare e non decomprimere mai. Noi facciamo altro, infatti con la feature `gzip` attiva, `reqwest` decomprime in streaming e **rimuove** l'header `Content-Encoding`, quindi il nostro `reject_content_encoding` per il `gzip` non scatta mai: resta a guardia delle codifiche che non sappiamo gestire, `br` e `zstd`. La bomba viene fermata dal tetto `max_input_bytes` applicato nel ciclo dei chunk, oppure dall'errore di decodifica su uno stream troncato.

L'esito coincide (`Refused` in entrambi i casi) il meccanismo no. Riteniamo la nostra scelta difendibile e forse migliore: rifiutare ogni risposta con `Content-Encoding: gzip` significherebbe rifiutare buona parte del web, dato che è compressione di trasporto standard e non un attacco. La decompressione limitata in streaming dà il contenuto e la protezione, e soddisfa il requisito sull'assenza di allocazioni non limitate meglio di un rifiuto cieco: l'espansione si ferma prima di essere allocata, non dopo.

/mime/scripted-pdf. L'atteso è «*Detect the JavaScript action and either strip it or rasterize all pages*». Noi rifiutiamo il documento, che non è nessuna delle due opzioni. Togliere un'azione da un PDF vorrebbe dire riscriverlo dall'interno, e trasformarlo in immagini vorrebbe dire incorporare un motore di rendering: in entrambi i casi si aggiungerebbe una libreria pesante a cui dare in pasto input ostile, cioè proprio ciò che un sanitiser dovrebbe evitare. È la risposta conservativa, e la sezione 9 ci esonera dall'analisi dinamica. Il rifiuto colpisce comunque **solo i PDF che contengono davvero un marcatore di esecuzione**: tutti gli altri passano intatti.

Un caso merita una nota: [/html/echo-headers](#) è scritto per un sanitiser esposto come servizio HTTP, che riceve una richiesta con credenziali e potrebbe inoltrarle a monte. Una CLI non riceve header da nessun client, quindi il nostro test dimostra che il client non **origina** credenziali.

7. Valutazione sperimentale

7.1 Metodologia

Tutte le misure provengono da una singola esecuzione di [scripts/benchmark.py](#) su build di release, su una macchina a **16 core**, con **5 ripetizioni** per punto. I dati grezzi sono in [scripts/results.json](#), le figure in [scripts/figures/](#).

Tre scelte metodologiche vanno dichiarate perché influenzano l'interpretazione.

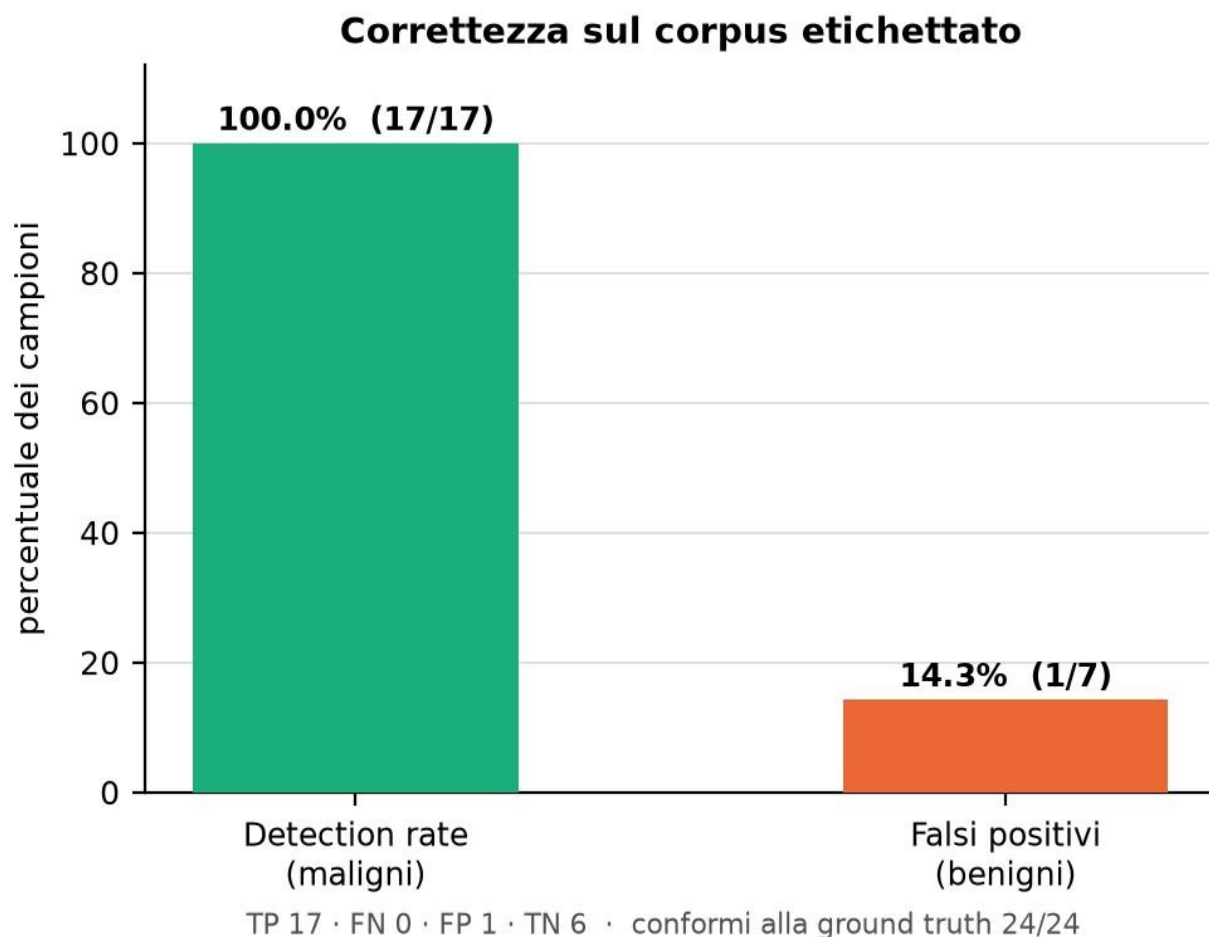
La prima ripetizione di ogni serie viene scartata. Dalla seconda in poi gli input sono nella page cache del sistema operativo, quindi si misura il tool invece del disco.

Si riporta la mediana, non la media. Le esecuzioni si somigliano quasi tutte, ma ogni tanto una dura molto di più (lo scheduler, un processo che parte in background) e non capita mai il contrario. La media si lascia trascinare da quei pochi valori alti, la mediana no.

Il tempo di parete include l'avvio del processo e la scrittura degli output. È la latenza che vede chi usa la CLI, ma significa che su input piccoli una frazione non trascurabile del tempo misurato non è sanitizzazione. Per questo la latenza per input si ricava da un lotto di copie diviso il numero di copie: misurare un singolo file da 16 KiB vorrebbe dire misurare soprattutto l'avvio del processo.

Gli input per prestazioni, scalabilità e memoria sono **pagine sintetiche generate dallo script**: la ripetizione dello stesso blocco, con la stessa proporzione di costrutti pericolosi a ogni dimensione. Serve a poter affermare che una curva sale perché il file è più grande, non perché è fatto in modo diverso.

7.2 Correttezza



Metrica	Valore
Veri positivi	17 su 17
Falsi negativi	0
Detection rate	100%
Veri negativi	6 su 7
Falsi positivi	1
False positive rate	14,3%
Conformità alla ground truth	24 su 24

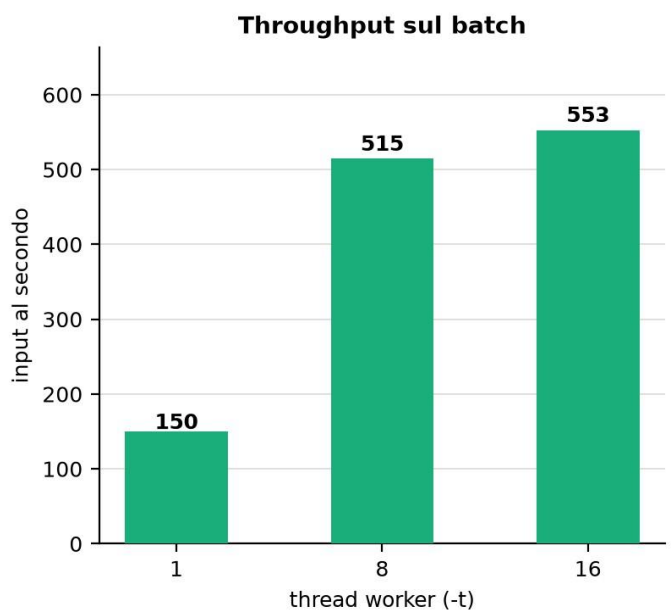
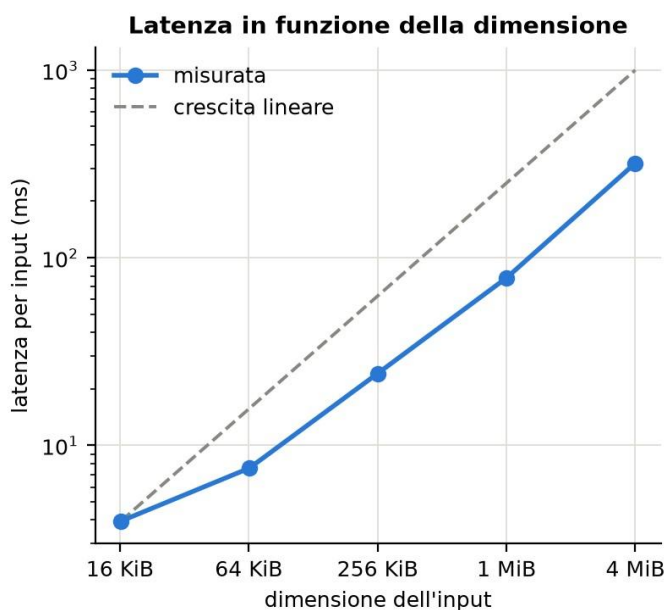
Il detection rate è del 100%: nessun campione maligno è passato inosservato, e tutte le regole attese sono comparse fra le azioni.

L'unico falso positivo è [benign/tricky-but-legit.html](#), ed è istruttivo. La pagina contiene un `data:` URI di una piccola immagine GIF reale e innocua. Sotto la policy di default `allow_data_uri` è `false`, quindi lo schema viene neutralizzato e la pagina risulta “toccata” pur essendo benigna.

Non è un difetto da correggere ma un **compromesso da riconoscere**: i `data:` URI sono un vettore XSS classico che aggira i controlli ingenui sugli schemi, e distinguere un `data:image/gif` legittimo da un `data:text/html` ostile richiederebbe di ispezionare il contenuto codificato. La policy permette di ammetterli con `allow_data_uri: true`, e in quella configurazione il falso positivo sparisce.

Con un solo falso positivo su sette benigni il tasso apparente è alto, ma il denominatore è piccolo: un corpus benigno più ampio darebbe una stima più stabile, ed è il primo ampliamento che suggeriremmo.

7.3 Prestazioni



Latenza per input in funzione della dimensione, a thread singolo:

Dimensione	Latenza	Throughput
16 KB	3,94 ms	4,00 MB/s
64 KB	7,56 ms	8,32 MB/s
256 KB	24,2 ms	10,36 MB/s
1 MB	78,1 ms	12,82 MB/s
4 MB	319 ms	12,54 MB/s

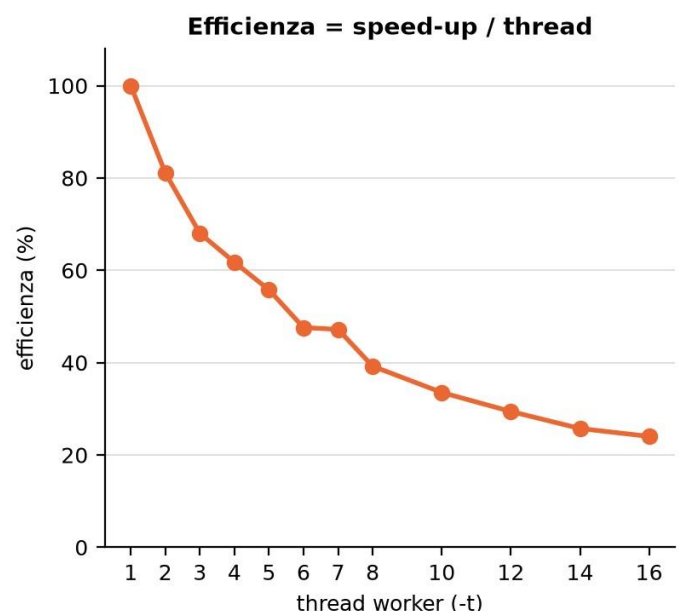
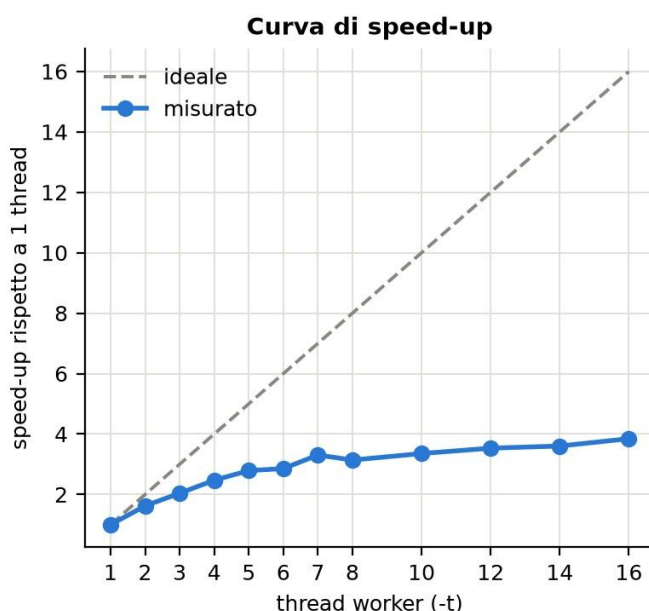
La latenza cresce **linearmente** con la dimensione: da 16 KB a 4 MB l'input si moltiplica per 253 e il tempo per 81. La crescita sublineare del tempo rispetto ai byte si spiega col throughput, che sale da 4 a 12,8 MB/s: sui file piccoli il costo fisso (avvio del processo, costruzione del runtime e apertura dei file) pesa proporzionalmente di più, e viene ammortizzato man mano che il documento cresce.

Il throughput si stabilizza intorno ai 12,5–12,8 MB/s fra 1 e 4 MB: è il regime in cui il costo fisso è trascurabile e si misura la velocità reale del rewriter in streaming.

Throughput su lotto, a parità di corpus:

Thread	Tempo	Input/s	MB/s
1	0,798 s	150,3	9,45
8	0,233 s	514,9	32,38
16	0,217 s	552,6	34,75

7.4 Scalabilità



Thread	Tempo	Speed-up	Efficienza
1	0,797 s	1,00×	100%
2	0,491 s	1,62×	81%
4	0,323 s	2,47×	62%
8	0,254 s	3,14×	39%
12	0,226 s	3,53×	29%
16	0,207 s	3,85×	24%

Lo speed-up è **reale ma sublineare**: 3,85× su 16 core, con l'efficienza che scende dal 100% al 24%.

Dove sta il collo di bottiglia. Non nel parser: la sanitizzazione è CPU-bound e indipendente fra input, quindi dovrebbe scalare. Le cause plausibili, in ordine di peso:

Il costo fisso non parallelizzabile. Avvio del processo, costruzione del runtime tokio e del client HTTP, scrittura del report finale avvengono una volta sola e in sequenza. Con un lotto che a 16 thread dura 207 ms, una frazione fissa di qualche decina di millisecondi impone da sola un tetto secondo la legge di Amdahl. È verosimilmente la causa principale, ed è anche la più facile da confermare: basterebbe misurare lo stesso corpus replicato dieci volte per vedere l'efficienza risalire.

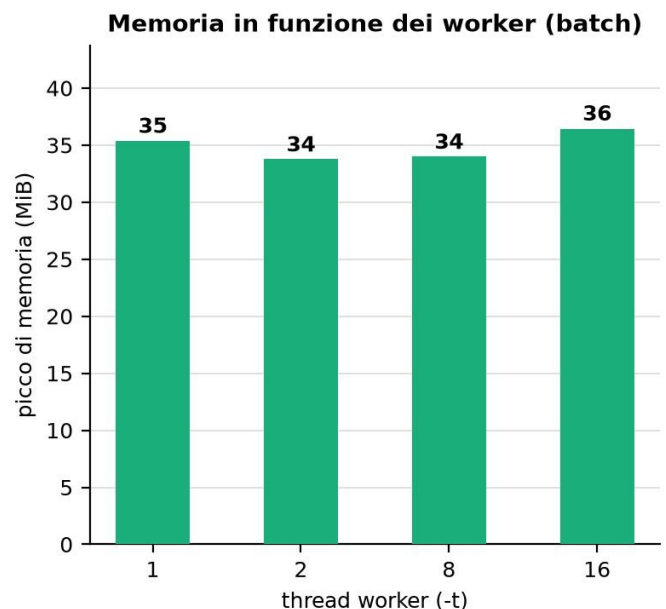
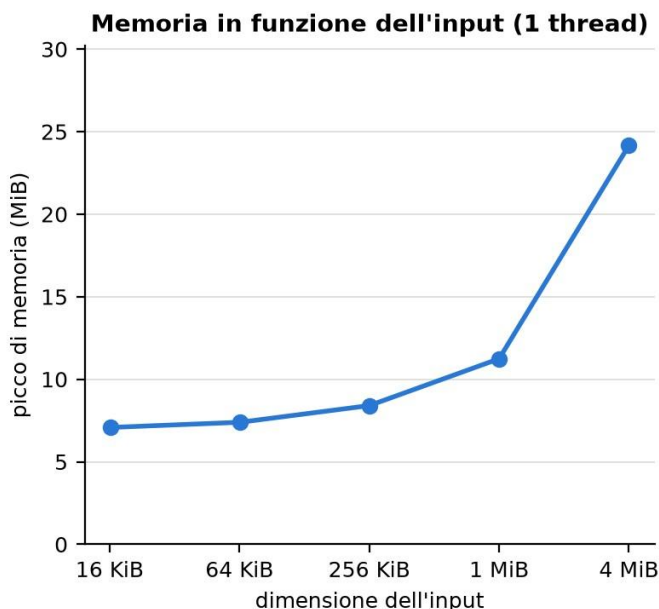
La contesa sulla coda. Tutti i worker prendono lo stesso **Mutex** per estrarre un job. La sezione critica è minima, ma con 16 thread e input piccoli (il corpus ha file di poche centinaia di byte) la frequenza di accesso è alta rispetto al lavoro utile per elemento.

La granularità del lavoro. Il corpus è fatto di 24 file piccoli. Un lotto con file più grandi darebbe più lavoro per acquisizione del lock e curve migliori: è il tipo di misura che un ampliamento del corpus permetterebbe.

L'I/O di scrittura. Ogni worker scrive il proprio output su disco, e le scritture concorrenti sulla stessa directory competono per il file system.

Si nota anche un'inversione fra 7 e 8 thread (3,30× contro 3,14×) che attribuiamo a rumore di scheduling: è nell'ordine di grandezza della variabilità residua dopo la mediana su cinque ripetizioni, e non a un fenomeno strutturale.

7.5 Uso di memoria



Per dimensione dell'input, a thread singolo:

Input	Picco residente
16 KB	7,43 MB
64 KB	7,75 MB
256 KB	8,81 MB
1 MB	11,79 MB
4 MB	25,34 MB

È il dato che conferma la scelta dello streaming. **L'input cresce di 253 volte, il picco di memoria di 3,4 volte.** Se il rewriter costruisse un albero DOM la memoria crescerebbe proporzionalmente al documento, e con un fattore di espansione tipicamente superiore a uno, perché ogni nodo porta con sé la propria struttura.

La base di circa 7,4 MB è il costo dell'eseguibile, del runtime tokio e dell'allocatore: su input piccoli domina, ed è il motivo per cui la curva parte piatta.

La crescita che si osserva sui file grandi è comunque attesa: il documento viene letto interamente in memoria prima di essere riscritto, quindi il picco include i byte di ingresso e quelli di uscita. Lo streaming elimina l'albero, non il buffer.

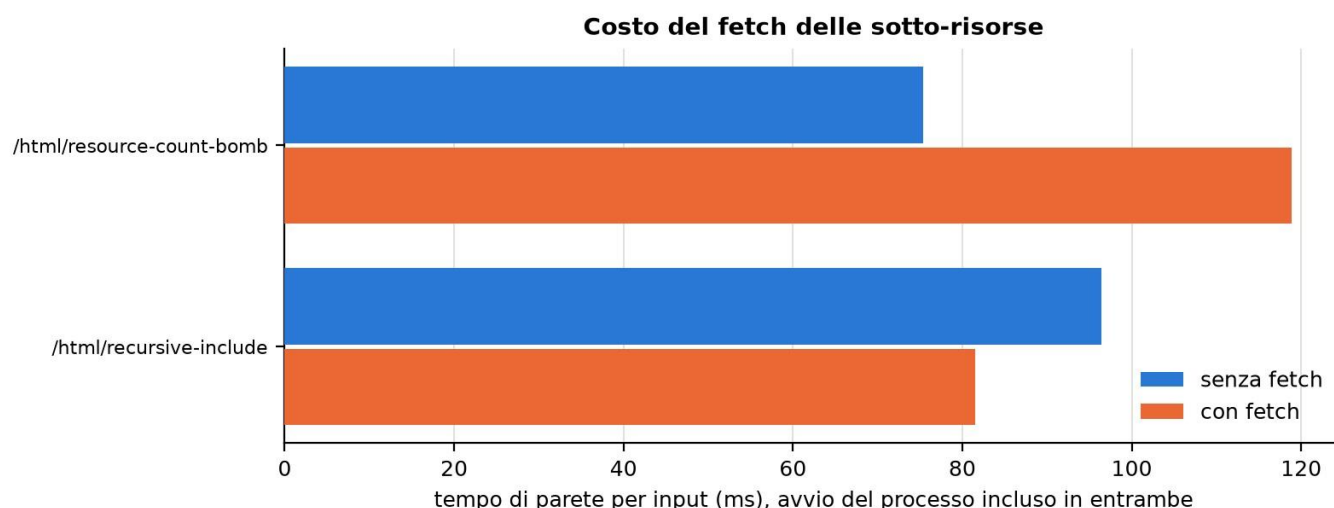
Per numero di worker, a parità di corpus:

Thread	Picco residente
1	37,08 MB
2	35,41 MB
8	35,65 MB
16	38,20 MB

La memoria è **sostanzialmente piatta**: fra il minimo e il massimo la differenza è di 2,8 MB su circa 36, cioè meno dell'8%, e non è nemmeno monotona rispetto ai thread (il valore più alto è a 16 worker ma il più basso è a 2, non a 1). È il profilo di una misura dominata dal rumore, non da una tendenza. È la conferma che **WorkerContext** condivide invece di duplicare: policy, allow-list e client HTTP esistono in una copia sola, e ogni worker aggiunge solo il proprio stack e il buffer del documento in lavorazione.

Se ogni worker avesse una propria copia della configurazione e del client, la curva salirebbe linearmente con i thread, invece non lo fa.

7.6 Il costo del fetch delle sotto-risorse



Il confronto con e senza `fetch_subresources`, contro il container evil-origin:

Rotta	Senza	Con	Δ	Sotto-risorse scaricate
/html/resource-count-bomb	75,3 ms	118,9 ms	+43,6 ms	20
/html/recursive-include	96,4 ms	81,6 ms	-14,8 ms	1

Sulla prima rotta il costo è netto e interpretabile: venti sotto-risorse scaricate per **+43,6 ms**, cioè circa 2,2 ms per risorsa su rete locale. La pagina ne dichiara cinquanta e il crawl si ferma a venti: è il tetto `max_fetch_requests` che agisce, ed è la dimostrazione sperimentale che il limite richiesto dalla sezione 3 funziona.

Sulla seconda rotta la differenza è **negativa**, il che è ovviamente impossibile come costo reale: la pagina scarica una sola sotto-risorsa, e il costo di quella singola richiesta è sotto la soglia di rumore della misura. Il valore va letto come “indistinguibile da zero”, non come un guadagno.

Il caso è comunque significativo per un'altra ragione: `recursive-include` si riferenzia da sola con lo **stesso URL**, e il crawl la scarica **una volta sola**. A spezzare il ciclo è l'insieme `visited`, non il tetto di profondità, che non fa in tempo a intervenire. Entrambi soddisfano il requisito di non bloccarsi, ma il meccanismo che agisce non è quello che il nome dello scenario suggerisce.

Un'avvertenza sulla configurazione. Per queste due misure la policy del banco devia dal default su due campi, allineandosi a `evil_origin_test.rs: max_fetch_depth: 2` e `max_total_fetch_bytes: 100 MB`, quest'ultimo perché a fermare il crawl sia il tetto sul numero di richieste e non quello sui byte. I tempi riportati non sono quindi quelli della configurazione di produzione.

8. Limiti noti ed estensioni possibili

8.1 Limiti dell'analisi

Regole locali all'elemento. Il modello a streaming non permette trasformazioni che richiedono una visione globale dell'albero.

CSS solo autonomo. `sanitise_css` si applica ai fogli di stile serviti come `text/css`. I blocchi `<style>` e gli attributi `style=` dentro l'HTML **non vengono ispezionati**: un `expression()` scritto in uno `<style>` inline sopravvive.

Sottoinsieme di attributi. Il rewriter ispeziona sei attributi che portano URL. Un `srcset` o un `background` con un `data:` URI non verrebbe intercettato e nemmeno rilevato dai nostri test, che usano lo stesso elenco.

8.2 Limiti del motore

I panic dei worker vengono assorbiti in silenzio. Il `join()` finale ignora l'esito dei thread: un panic dentro una libreria di terze parti non farebbe cadere il programma, il che è desiderabile, ma il job corrispondente sparirebbe dal report senza che nessuno lo segnali. La correzione è propagare l'esito della `join` e registrare un `JobReport` con stato `Error`.

La funzione `crawl_subresources` non si applica ai riferimenti relativi. Un file che referencia `https://cdn-sospetto.test/app.js` vede quel link ispezionato ma non seguito. Il cambiamento minimo, costruire una base `file://`, coprirebbe i riferimenti assoluti; seguire anche i relativi richiederebbe un ramo di lettura da disco con contenimento del path traversal.

Duplicazione della logica di ispezione. L'albero di decisione per tipo esiste in due copie, in `worker::process_one` e in `crawl_subresources`. Una modifica a una regola va replicata in due punti.

Un'azione persa. Quando il crawl sfiora il budget di tempo, l'azione che lo registra viene prodotta ma poi scartata insieme a tutte le altre, perché il `return` che segue costruisce un `JobReport::refused`, che azzerava le azioni.

8.3 Limiti della valutazione

Il corpus benigno è piccolo. Sette campioni rendono il false positive rate una stima con un denominatore troppo basso. È l'ampliamento più utile.

La scalabilità è misurata su input piccoli. Il corpus reale ha file di poche centinaia di byte, il che amplifica il peso del costo fisso e della contesa. Ripetere le curve su un batch di file grandi separerebbe i due effetti.

Il picco di memoria è una stima per difetto. Il campionamento avviene ogni 3 ms: un picco molto breve può sfuggire.

8.4 Estensioni

In ordine di rapporto fra valore e costo:

1. **Sanitizzazione dei blocchi `<style>` e degli attributi `style=`:** collegare il ripulitore CSS al rewriter
2. **Ampliamento del corpus benigno:** dieci o quindici pagine reali per una stima credibile dei falsi positivi
3. **Crawl per i file locali:** nella forma minima, base `file://` e soli riferimenti assoluti
4. **br e zstd:** `reqwest` li supporta dietro feature, la logica di rifiuto per ciò che resta fuori non cambia
5. **Unificazione dell'albero di decisione:** una funzione sola condivisa fra worker e crawl

9. Conclusioni

Il progetto realizza tutti i requisiti funzionali della sezione 3 e i vincoli non funzionali della sezione 4, con una copertura di test su tre livelli e una valutazione sperimentale che misura le quattro grandezze richieste dalla sezione 7.

I risultati che riteniamo più significativi sono tre:

- **il detection rate del 100% con un solo falso positivo**, e quel falso positivo è un caso di confine documentato e configurabile, non un difetto di implementazione;
- **la memoria che cresce di 3,4 volte a fronte di un input che cresce di 253 volte**, che è la conferma sperimentale della scelta di parsing in streaming richiesta dalla sezione 5.2;
- **la memoria piatta al crescere dei worker**, che conferma che lo stato immutabile è condiviso e non duplicato.

Il risultato meno soddisfacente è lo speed-up sublineare, $3,85\times$ su 16 core. Non lo consideriamo un difetto strutturale del motore ma una conseguenza della granularità del carico di prova, e abbiamo indicato in sez. 7.4 la misura che permetterebbe di confermarlo.

Sul piano della programmazione di sistema, il progetto sostiene la tesi che le garanzie di Rust si prestano bene a questa classe di problemi: **zero blocchi unsafe**, assenza di data race garantita dal sistema dei tipi, e un modello di errore che rende impossibile ignorare un fallimento. Su uno strumento il cui input è per definizione ostile, sono garanzie che valgono più di qualche punto percentuale di prestazioni.